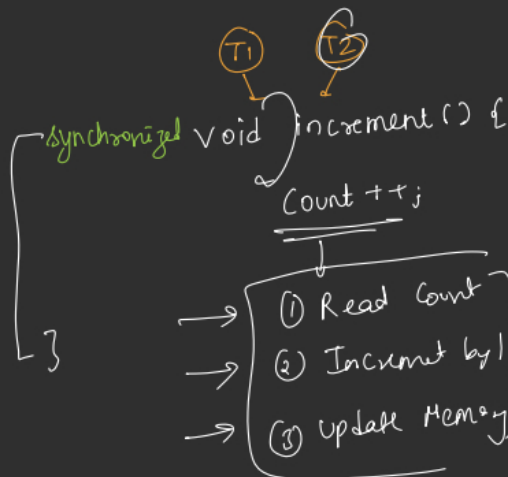


Synchronized



Critical
Section

```
class Counter {  
    int count = 0;  
    void increment() {  
        count++;  
    }  
}
```

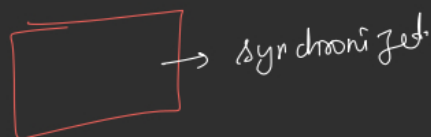


- ① T1 acquires lock.
- ② T2 tries to lock → block state.
- ③ T1 will perform task.
- ④ As soon T1 → exits T2 can enter

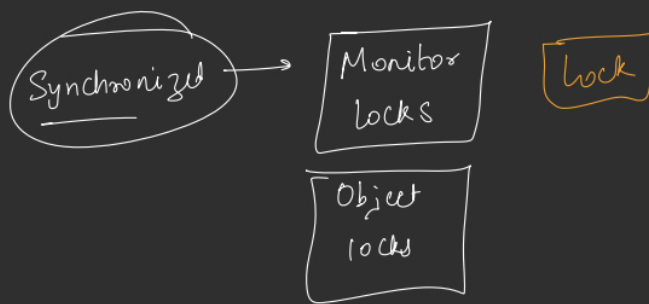
Why do we need Synchronized?

- ① To protect shared data.
- ② To make any opⁿ atomic.
- ③ To ensure visibility.
- ④ To prevent re-ordering.

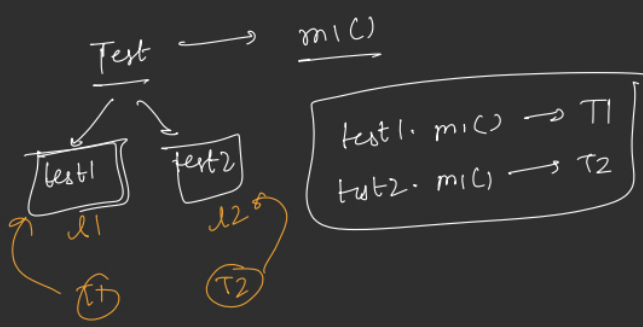
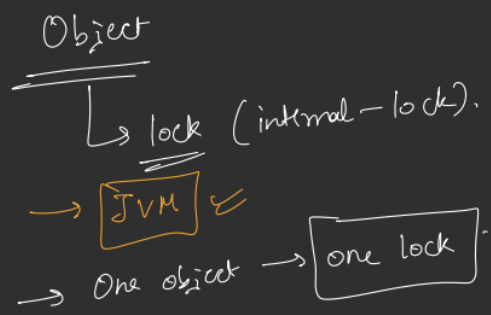
{



}



Class Count {
 Synchronized void increment() {
 count++;
 }
}



Synchronized block //

{
 []
 }
 synchronized (this) {
 []
 }

Custom lock objects

How locks work Internally.

Object (obj)

↳ lock : {
 ownerThread : null
 isLocked : false
 waitingQueue : []
}

synchronized (obj) {

// Critical section. }

if (obj.lock.isLocked == false) {
 obj.lock.isLocked = true
 obj.lock.ownerThread = t1
}

else {
 // someone is already inside
 obj.lock.waitingQueue.add(t2);
}

Synchronized

↳ overhead

→ slow

Optimistic locking